eman ta zabal zazu

Universidad del País Vasco
Euskal Herriko Unibertsitatea

Sistemas de Gestión de Seguridad de Sistemas de Información

Sistema Web

Diego Pomares

Ivan Salazar

Lou Marine Gómez

David Miguez

29/10/2025

Ingeniería Informática de Gestión y Sistemas de Información

ÍNDICE

1. Introducción....3
2. Segunda Auditoría.... 4
2.1. ZAP.... 4
3. Vulnerabilidades....5
3.0. Implementación de init.php & PHP.8.2.... 5
3.2. Medidas ante el Bruteforcing....5
3.2. Implementación de Token CSRF....5
3.3. Samesite Attribute & HttpOnly flag....6
3.4. Headers....6
3.5. CSP Headers.... 7
3.6. Directory Browsing.... 7
4. Bibliografía.... 8

1. Introducción

En este documento se recogen todos los cambios que se realizaron en la web para corregir las vulnerabilidades que ZAP detectó durante las pruebas. A lo largo del proceso se ajustaron cabeceras HTTP, se mejoró la configuración del servidor Apache, se reforzó el manejo de sesiones y se añadieron medidas para prevenir ataques de fuerza bruta.

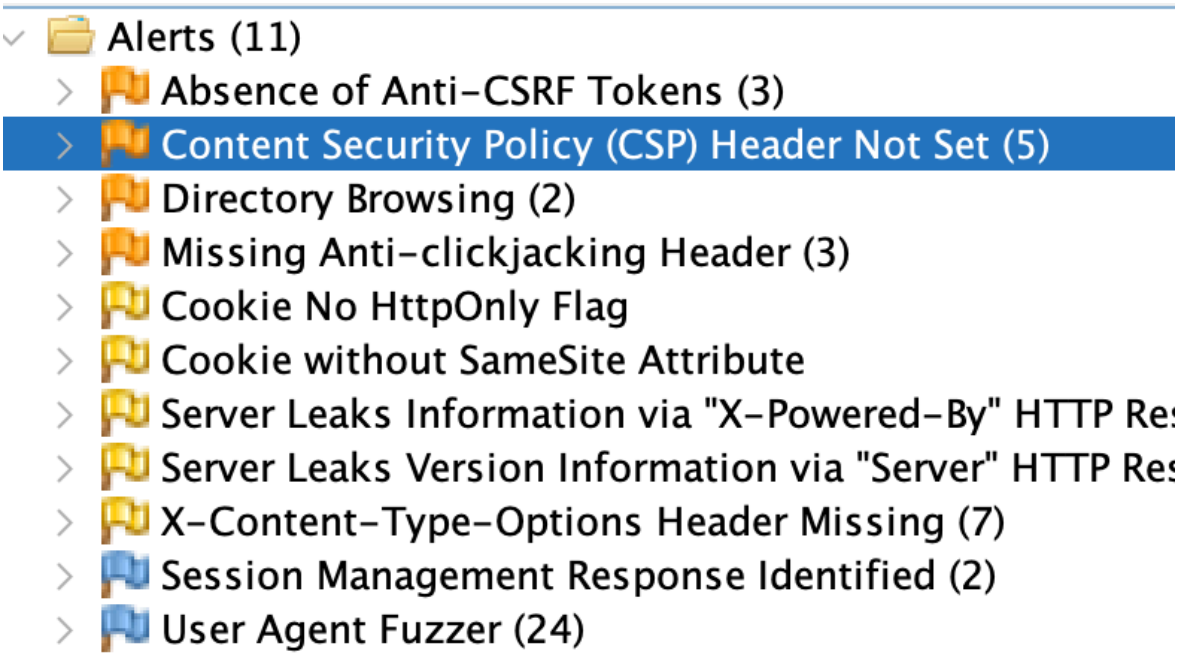
El objetivo fue reducir al mínimo la exposición de información sensible y aumentar la seguridad general del sitio sin afectar su funcionamiento normal. A continuación, se muestran las correcciones aplicadas y cómo cada una ayudó a cerrar los puntos débiles encontrados.

Toda vulnerabilidad crítica que no sale como inyecciones SQL se han cubierto en la auditoría anterior (también incluida en el repositorio).

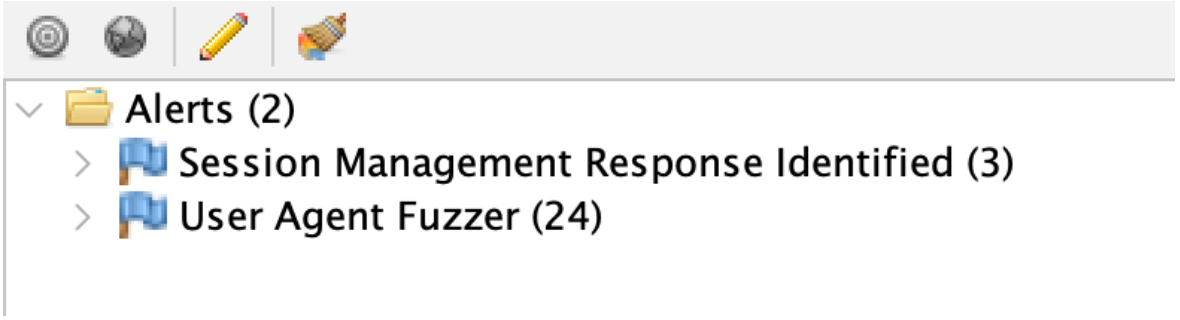
2. Capturas de ZAP

2.1. ZAP

Antes



Después



3. Vulnerabilidades

3.0. Implementación de init.php & PHP.8.2

Se ha creado un nuevo archivo init.php que se le llama en cada .php el cual participa en el arreglo de una serie de vulnerabilidades que se irán tratando. Además, ahora utilizamos más recientes de php.

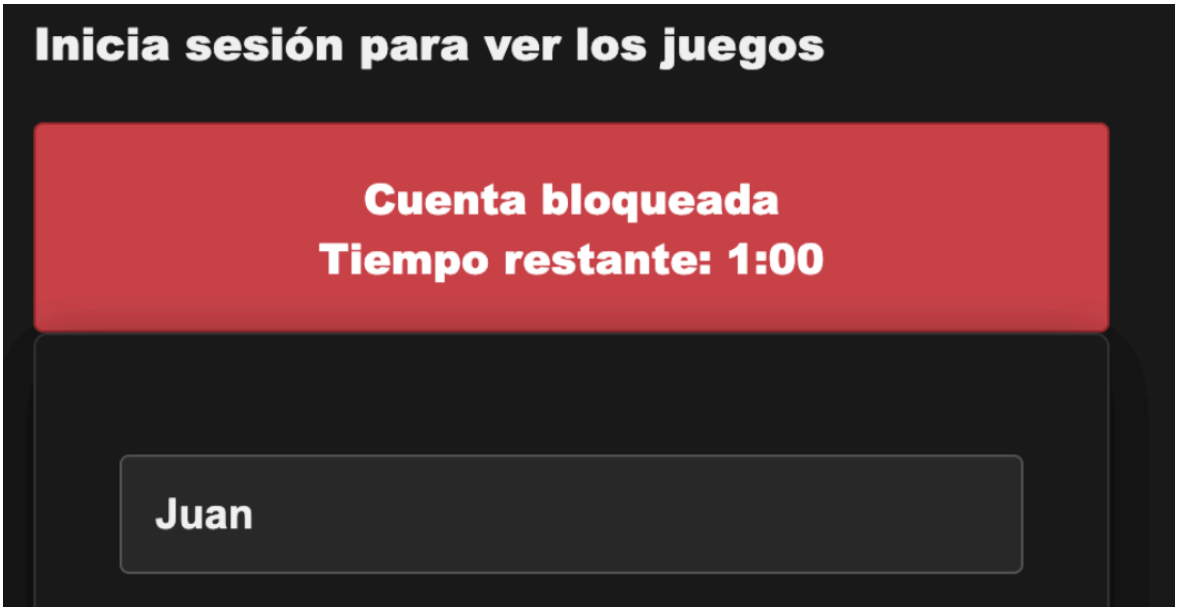
```
[√Multi 1:24:22 ~/Documents/CyberSec/Proyectos/SGSSi-Proyecto $ cat app/init.php
<?php
if (session_status() === PHP_SESSION_NONE) {
    session_set_cookie_params([
        'lifetime' => 0,              // session cookie
        'path' => '/',
        'domain' => '',
        'secure' => isset($_SERVER['HTTPS']), // only send over HTTPS
        'httponly' => true,           // mitigate XSS
        'samesite' => 'Strict'        // or 'Lax' if you use external redirects
    ]);
    session_start();

    if (empty($_SESSION['csrf_token'])) {
        $_SESSION['csrf_token'] = bin2hex(random_bytes(32));
    }

}
require_once 'connect.php';
```

3.1. Medidas ante Bruteforcing

El sistema aplica una doble protección ante intentos fallidos de inicio de sesión. Cada IP puede fallar hasta 5 veces antes de quedar bloqueada durante 1 minuto, lo que evita ataques de fuerza bruta y la enumeración de usuarios. Si el usuario existe, también se lleva un conteo independiente: tras 5 fallos, su cuenta queda bloqueada por el mismo tiempo. Los contadores de IP y usuario se incrementan con cada error y se restablecen al lograr un acceso correcto.



Esto se ha implementado con código en login_action.php y cambios en el .sql los cuales se pueden ver en detalle en el commit “Evitar ataques de fuerza bruta al login de la aplicación” de Github del 7 de noviembre de 2025, el cual es algo largo para mostrar aqui.

3.2. Implementación de Token CSRF

Init.php ahora incluye una generación de un Token CSRF para que cada vez que se cree una sesión se genere un Token único.

```
if (empty($_SESSION['csrf_token'])) {
    $_SESSION['csrf_token'] = bin2hex(random_bytes(32));
}
```

Luego, al mandar los forms se manda como input oculto para luego verificarlo dentro de login_action.php y register_action.php para que salte un error 403 cuando no se sea la correcta.

```
<input type="hidden" name="csrf_token" value="<?php echo htmlspecialchars($_SESSION['csrf_token']); ?>">
```

```
if ($_SERVER['REQUEST_METHOD'] === 'POST') {
    if (!isset($_POST['csrf_token']) || !hash_equals($_SESSION['csrf_token'], $_POST['csrf_token'])) {
        http_response_code(403);
        die("CSRF validation failed.");
    }
}
```

3.3. Samesite Attribute & HttpOnly flag

Al poner el flag httponly en true se evita que las cookies sean accesibles desde JavaScript, reduciendo el riesgo de robo por XSS. Con el atributo samesite en strict, las cookies no se envían en peticiones cruzadas, evitando ataques CSRF.

```
session_set_cookie_params([
    'lifetime' => 0,              // session cookie
    'path' => '/',
    'domain' => '',
    'secure' => isset($_SERVER['HTTPS']), // only send over HTTPS
    'httponly' => true,           // mitigate XSS
    'samesite' => 'Strict'        // or 'Lax' if you use external redirects
```

3.4. Headers

Estas directivas en apache2.conf refuerzan la seguridad del sitio. X-Frame-Options con valor DENY evita que la página se cargue dentro de iframes, protegiendo frente a ataques de clickjacking. X-Content-Type-Options con nosniff impide que el navegador interprete tipos de contenido distintos a los declarados, reduciendo riesgos de ejecución

de código malicioso. X-XSS-Protection activa el filtro del navegador contra ataques de Cross-Site Scripting.

```
<IfModule mod_headers.c>
    Header always set X-Frame-Options "DENY"
    Header always set X-Content-Type-Options "nosniff"
    Header always set X-XSS-Protection "1; mode=block"
    Header always set Referrer-Policy "strict-origin-when-cross-origin"
    Header always set Permissions-Policy "geolocation=(), microphone=()"
    Header always set Content-Security-Policy "default-src 'self'; scrip
to 'self';"
    Header always unset X-AspNet-Version
    Header always unset X-AspNetMvc-Version
    Header always unset X-Powered-By
    Header always unset Server
</IfModule>
```

Además se ha intentado esconder los headers X-Powered-By & Server para evitar que revelen info sobre OS/Apache/PHP pero no ha funcionado así que se han añadido las siguientes líneas en el Dockerfile, lo cual parece haber funcionado.

```
#Esconde la versión de PHP
RUN echo "expose_php=Off" > /usr/local/etc/php/conf.d/security.ini

#Esconde los leaks del campo Server en los requests
RUN echo "ServerTokens Prod" >> /etc/apache2/conf-enabled/security.conf && \
    echo "ServerSignature Off" >> /etc/apache2/conf-enabled/security.conf && \
    echo "Header always unset Server" >> /etc/apache2/conf-enabled/security.con
```

3.5. CSP Headers

Los Headers de CSP evitan que se trate con archivos que no se quieran (ya sean scripts, .css, etc). Su implementación ha dado una serie de problemas dado que parte del código de JS estaba escrito dentro del html en lugar de un fichero aparte lo cual generaba problemas a la hora de llevar a cabo las redirecciones en login y register (y la cabecera bloqueaba el script inline al ser muy estricta), así que estos scripts se han separado y colocado dentro del directorio /js, lo cual es más limpio y ordenado. Esta es la línea de los headers CSP:

Header always set Content-Security-Policy "default-src 'self'; script-src 'self'; connect-src 'self'; object-src 'none'; frame-ancestors 'none'; base-uri 'self'; style-src 'self'; form-action 'self'; navigate-to 'self';"

Además se ha tenido que añadir el módulo en apache2.conf

```
# Habilitar mod_rewrite & Headers
RUN a2enmod rewrite
RUN a2enmod headers
```

3.6. Directory Browsing

En este bloque se ha deshabilitado la opción Indexes con *Options -Indexes*, lo que impide que el servidor muestre el listado de archivos del directorio cuando no hay un archivo índice presente. Este cambio evita el directory browsing, protegiendo así información sensible o estructura interna del sitio.

```
<Directory /var/www/>
    Options -Indexes +FollowSymLinks
    AllowOverride All
    Require all granted
    # Reduce access to dotfiles
    <FilesMatch "^\.">
        Require all denied
    </FilesMatch>
</Directory>

<Directory /var/www/html>
    Options -Indexes +FollowSymLinks
    AllowOverride All
    Require all granted
</Directory>
```

4. Bibliografía

- Apuntes de Mikel
  https://github.com/mikel-egana-aranguren/EHU-SGSSI-01/tree/main/Seguridad_web
- Mozilla Developer Network
  https://developer.mozilla.org/en-US/docs/Web/HTTP/Guides/CSP
  https://developer.mozilla.org/en-US/docs/Web/HTTP/Headers
- web.dev
  https://web.dev/articles/security-headers?hl=es
- OWASP
  https://owasp.org/www-project-top-ten/
- ChatGPT
  https://chatgpt.com/